

はいれつ 配列

はいれつ I. 配列 (Array) の基本

たくさんのデータをバラバラに管理するのは大変です。そこで使うのが「Arrayオブジェクト」という、^{ふくすう}複数のデータをひとつにまとめて管理できるリストのような道具です。日本語では、「^{はいれつ}配列 (はいれつ)」と呼びます。

^{はいれつ}配列 (はいれつ) は「ならんだ箱」

たくさんのデータを、ひとつのグループとしてまとめたいときに、「^{はいれつ}配列」を使います。学生 みなさんには、「^{ばんごう}番号がついたロッカー」や「^{ばんごう}番号がついた部屋が並んでいるマンション」をイメージすると分かりやすくなります！

[!TIP] 漢字のヒント 「配」 は「くばる」、「列」 は「ならぶ」という意味です。みんなが並んでいる様子をイメージすると、覚えやすいですよ！

コンストラクタ

オブジェクトを利用するには、^{えんざんし}new演算子 と **コンストラクタ** を使って、新しいオブジェクトを生成します。

```
a = new array(1,2,3);
```

ここでは、`array()` がコンストラクタです。コンストラクタはオブジェクトを生成するための特別な関数で、オブジェクトと同じ名前をしています。引数はオブジェクトによって違いますが、`array()` の場合は、^{なかみ}中身をひとつひとつ、`,` (カンマ) で区切って並べます。

また、^{はいれつ}配列は、次で説明するように、^{かく}角カッコを使って、作成することもできます。

[] (^{かく}角カッコ)

JavaScriptで^{はいれつ}配列 (はいれつ) を使うとき、必ず出てくるのが `[ ]` (^{かく}角カッコ) です。

大きく分けて、**2つの使い方**があります。

^{はいれつ}配列を「作る」ときの `[ ]`

→ 「新しいマンションを建てる」イメージ

一番はじめに、^{なかみ}中身をまとめて入れるときに使います。

```
const colors = ["赤", "青", "黄 (き) "];
```

- `[` : ここからマンションが始まります。
- `,` : 部屋（へや）と部屋の間の「かべ」です。
- `]` : ここでマンションが終わります。

なかみ

中身を「指定する」ときの `[ ]`

→ 「ルームナンバーで呼ぶ」イメージ

作ったあとで、「〇番目のデータがほしい！」というときに使います。

```
console.log(colors[0]); // 「赤」が出てくる
```

ここが大事！「0番」から始まるルール

マンションの「1号室（いちごうしつ）」は、プログラミングでは `[0]` と書きます。

- 1つめ: `colors[0]`
- 2つめ: `colors[1]`
- 3つめ: `colors[2]`

[!TIP] 覚え方：プログラミングの世界では、「まだ一歩（いっぽ）も動いていない状態」を `0` と考えます。だから最初が `0` なんです。「1つめは `0`（ゼロ）！」と心の中で3回唱えてください。これだけで、バグが半分に減りますよ！

なかみ

中身を「書きかえる」ときの `[ ]`

→ 「ロッカーの中身を 入れ替える」イメージ

特定のロッカー番号を指定して、中身を新しくすることもできます。

```
const snacks = ["チョコ", "グミ"];

snacks[1] = "アイス"; // 1番ロッカー（2つめ）のグミを、アイスに変える

console.log(snacks); // ["チョコ", "アイス"]
```

この「中身を「書きかえる」ときの `[ ]`」を使えば、最初に空の配列を作って、あとから、中身を入れることもできます。

```
const snacks = new array();
snacks[0] = "チョコ";
snacks[1] = "グミ";
```

[!NOTE] ここでは`snacks` という配列を、`const`で宣言しているのに、配列の中身を書きかえています。このことを不思議に思った人はいますか？ その人は、とてもすごい人です。

しかし、これはOKなんです。なぜなら、`const`を使って宣言されている`snacks`は、配列の中身ではなく、配列という入れ物のほうだからです。ロッカーの中身を入れ替えても、ロッカーそのものはもとのままで、少しも変わっていません。それと同じです。

まとめ：[] の役割

使い方	やっていること	たとえ話
<code>[a, b, c]</code>	配列を作る	マンションを建てる
<code>[0]</code>	データを取り出す	部屋をノックする
<code>[0] = "x"</code>	データを入れ替える	住人を入れ替える

[!NOTE] よくあるミス：全角の [] を使ってしまうとエラーになります。必ず半角の [] を使いましょう！

配列の要素

配列のひとつひとつの箱の中に入っているデータを配列の「要素」と言います。

配列の要素の数は`length`プロパティで、知ることができます。オブジェクトのプロパティを取り出すときは、「. (ドット)」を使います。

- `fruits.length` → 今、箱がいくつあるか (3つ) 教えてくれます。

```
console.log(fruits.length); // 3 と表示 (ひょうじ) されます。
```

学生へのアドバイス

いかがでしょうか？「0から始まるリスト」のイメージはつかめましたか？

II. よく使うプロパティ・メソッド・演算子

配列には、中身进行操作するための「道具 (メソッド)」や「情報 (プロパティ)」、そして「特別な記号 (演算子)」がたくさんあります。

学生の方に特に覚えてほしい、「これだけは外せない！」という便利なリストをまとめました。

よく使うプロパティ（配列の情報）

プロパティは、^{はいれつ}配列が持っている「データ」のことです。

length（レングス）

^{はいれつ}配列の中に、**今いくつデータが入っているか**を教えてください。

```
const students = ["マリア", "ジョン", "リー"];

console.log(students.length); // 3 が出る
```

- **イメージ**：列に並んでいる人の数をパッと数えるリーダーのような存在です。

なぜ大切なの？

^{はいれつ}配列の .length プロパティを使うことで、「データが今いくつあるか」を常に知ることができます。これにより、データの数が増えたり減ったりしても、プログラムを^か書き直さずに柔軟に^{じゅうなん}対応できるようになります。

覚えたい便利なメソッド（道具）

^{はいれつ}配列操作のメソッド

^{はいれつ}配列には、中身を操作するための便利なメソッドがたくさんあります。

^{だいひょうてき}代表的なメソッドを、「^{はいれつ}元の配列が変わるかどうか」と「どこを操作するか」に注目して、まとめて解説します。

^{はいれつ}配列を「^{でんしゃ}電車の車両」や「^{れつ}人の列」に例えて説明するのが一番わかりやすいです。

push（プッシュ）：おしりに追加する

一番最後（右側）に、新しいデータを追加します。

- **イメージ**：電車の最後に、新しい^{しゃりょう}車両をつなげる。
- **元の配列**：^{はいれつ}変わります。

```
const train = ["車両（しゃりょう）A", "車両（しゃりょう）B"];

train.push("車両（しゃりょう）C");

console.log(train); // ["車両（しゃりょう）A", "車両（しゃりょう）B", "車両（しゃりょう）C"]
```

pop (ポップ)：おしりをひとつ消す

一番最後（右側）のデータを1つ取り除きます。

- イメージ：電車の最後の車両^{しゃりょう}を切り離^{はな}す。
- 元の配列^{はいれつ}：変わります。

```
const train = ["車両（しゃりょう）A", "車両（しゃりょう）B", "車両（しゃりょう）C"];

const last = train.pop();

console.log(last); // "車両（しゃりょう）C"（消えたもの）
console.log(train); // ["車両（しゃりょう）A", "車両（しゃりょう）B"]
```

shift (シフト)：先頭をひとつ消す

一番最初（左側・0番目）のデータを消して、残りのデータを前にずらします。

- イメージ：お店の行列で、先頭の人^{はいれつ}が店に入り、後ろの人が全員前にずれる。
- 元の配列^{はいれつ}：変わります。

```
const queue = ["1番の人", "2番の人", "3番の人"];

queue.shift();
console.log(queue); // ["2番の人", "3番の人"]（みんな前にずれた！）
```

unshift (アンシフト)：先頭にひとつ追加する

配列の一番最初（先頭）に、新しいデータを追加します。

- イメージ：行列の一番前に割り込み（わりこみ）をする感じです。
- 元の配列^{はいれつ}：変わります。

```
let queue = ["2番の人", "3番の人"];

// unshift を使って、一番前に「1番の人」を入れます
queue.unshift("1番の人");

// 中身を確認（かくにん）してみましょう
console.log(queue); // 結果：["1番の人", "2番の人", "3番の人"]
```

slice (スライス)：一部をコピーする

指定（してい）した場所だけを、別の新しい^{はいれつ}配列として取り出します。

- **イメージ**：写真を撮（と）るようなものです。景色（元の^{はいれつ}配列）はそのまま、必要な部分だけを切^きり取ります。
- **元の^{はいれつ}配列**：変わりません（ここが重要！）。

```
const members = ["A", "B", "C", "D"];

// 1番目から3番目の「手前」までコピー
const sub = members.slice(1, 3);

console.log(sub);      // ["B", "C"]
console.log(members);  // ["A", "B", "C", "D"] （元の配列（はいれつ）はそのまま！）
```

splice（スプライス）:好きな場所にデータを出し入れする

^{はいれつ}配列の **好きな場所** のデータを消したり、新しく入れたりします。

- **イメージ**：「手術（しゅじゅつ）」です。悪い部分を切り取ったり、新しい部品を埋（う）め込んだりします。
- **元の^{はいれつ}配列**：変わります。

削除する例^{れい}

splice のうしろのカッコ（）には、2つの数字を書きます。

^{はいれつ}配列名.splice(^{ばんごう}スタートする番号, 消したい数);

- 1つめの数字：何番目（インデックス）から消し始めるか？
- 2つめの数字：そこから何個（いくつ）消すか？

```
const fruits = ["りんご", "バナナ", "オレンジ", "メロン"];

// 「バナナ」を消してみよう
// バナナは [1] 番目（2つめ）です。
fruits.splice(1, 1);

// 中身を確認（かくにん）します
console.log(fruits);
// 結果：["りんご", "オレンジ", "メロン"] （バナナがいなくなりました！）
```

入れ替^{い かえ れい}る例

splice のうしろのカッコ（）には、2つの数字と^{ようそ}要素を書きます。

はいれつ
配列名.splice(スタートする番号, 入れ替えたい数, 入れ替える要素);

- 1つめの数字：何番目（インデックス）から消し始めるか？
- 2つめの数字：そこから何個（いくつ）入れ替えるか？
- 入れ替える要素（2つ以上の場合は、`,`で区切って並べる）

```
const colors = ["赤", "青", "緑"];
// 1番目（青）と2番目を消して、代わりに「黄（き）」「白」を入れる
colors.splice(1, 2, "黄（き）", "白");
console.log(colors); // ["赤", "黄（き）", "白"]
```

includes（インクルーズ）:データがあるかないかを調べる

そのデータが配列の中にあるかないかを調べます。

- **イメージ**：探し物を見つける「警察犬」です。
- **答え**：`true`（ある）か `false`（ない）で返ってきます。

```
// カバンの中に「りんご」「ペン」「ノート」が入っています
const myBag = ["りんご", "ペン", "ノート"];

// 「ペン」が入っているかチェックします
const hasPen = myBag.includes("ペン");
console.log(hasPen); // 結果：true（あります！）

// 「バナナ」が入っているかチェックします
const hasBanana = myBag.includes("バナナ");
console.log(hasBanana); // 結果：false（ありません...）
```

delete 演算子（デリート）

特定の場所のデータを消しますが、**注意が必要**です。

- **注意**：データは消えますが、「箱（部屋）」は残ったままになります。部屋が「空（empty）」の状態になり、配列の長さ（length）も変わりません。

```
const snacks = ["チョコ", "グミ", "アメ"];
delete snacks[1];

console.log(snacks); // ["チョコ", <empty>, "アメ"]
console.log(snacks.length); // 3 のまま！（穴があいた状態）
```

[!IMPORTANT] 学生へのアドバイス： ^{はいれつ}配列からきれいにデータを消したいときは、`delete` ではなく `splice` を使うようにしてください。 `delete` は「穴」があくので、あまり使いません。

スプレッド^{えんざん}演算子 ... (てん・てん・てん)

^{はいれつ}配列の中身を ^{まほう}バラバラにして取り出す 魔法の記号です。

・ (どっと) を三つ並べて書きます。

- ・ **イメージ：** 袋に入ったお菓子を、テーブルの上に全部ザーッと広げる感じです。

```
const groupA = ["A", "B"];
const groupB = ["C", "D"];

// 2つの配列 (はいれつ) を合体させて、新しい配列 (はいれつ) を作る
const all = [...groupA, ...groupB];
console.log(all); // ["A", "B", "C", "D"]
```

比較 (ひかく) テーブル

種類	名前	場所	^{うご} 動き	元の配列 ^{はいれつ} は？
プロパティ	<code>length</code>	すべて	数を数える	変わらない
メソッド	<code>push</code>	最後	追加する	変わる
メソッド	<code>pop</code>	最後	消す	変わる
メソッド	<code>shift</code>	最初	消す (ずれる)	変わる
メソッド	<code>slice</code>	どこでも	コピーする	変わらない
メソッド	<code>unshift</code>	最初	足す	変わる
メソッド	<code>splice</code>	どこでも	^い ^か 入れ替える	変わる
メソッド	<code>includes</code>	すべて	あるか探す	変わらない
^{えんざん} 演算子	<code>delete</code>	すべて	穴をあけて消す	変わる (非 ^{すいしょう} 推奨)
^{えんざん} 演算子	<code>...</code>	すべて	中身を広げる	変わらない

学生へのアドバイス

「元の配列^{はいれつ}が変わってしまうメソッド (push, pop, shift)」を使うときは注意が必要です。もし、元のデータを壊 (こわ) したくないときは、`slice` を使ってコピーを作ってから作業する のがプロのコツですよ！

[!TIP] 「slice (スライス)」 は、料理でハムやチーズを薄 (うす) く切るのと同じ言葉です。「元の塊 (かたまり) から、一切れ分をもらう」と覚えると忘れません。

[!TIP] 「splice」と「slice」は名前が似ている！

- **splice**：手術（しゅじゅつ）して中身をいじる。
- **slice**：写真（しゃしん）のように一部を撮るだけ。

この違いを強調（きょうちょう）してあげると、学生も混乱（こんらん）しません。

III. 連想配列^{はいれつ}

「連想配列（れんそうはいれつ）」という言葉は、少し難しく聞こえますよね。簡単に言うと、「番号^{ばんごう}の代わりに、『名前（ラベル）』をつけた箱」のことで

JavaScriptでは「オブジェクト」の一種です。メソッドがなく、データだけをまとめたものです。

1. ふつうの配列^{はいれつ}との違い

普通の配列^{はいれつ}と、連想配列^{はいれつ}の違いを「ロッカー」でイメージしてみましょう。

- 普通の配列（Array）^{はいれつ}：番号^{ばんごう}がついたロッカー。
- 「0番のロッカー、1番のロッカー……」と、数字で中身を探します。
- 連想配列（Associative Array）^{はいれつ}：名前が書かれたラベル付きのロッカー。
- 「『名前』のロッカー、『国籍（こくせき）』のロッカー……」と、言葉で中身を探します。

2. 書き方^かと呼び方^{かた}

「名前（Key：キー）」と「中身（Value：値（あたい）」をセットにして作ります。

```
const animals = new array();
animals['dog'] = 'いぬ';
animals['cat'] = 'ねこ';
```

波かっこ`{}`を使って初期化することもできます。

```
// 連想配列（はいれつ）（オブジェクト）を作る
const user = {
  name: "田中",
  age: 20,
  country: "日本"
};
```

中身を取り出すには、ドット`.`やブラケット`[]`を使います。

```
// 中身を取り出す
console.log(user["name"]); // 「田中」が出る
console.log(user.name);    // これでも「田中」が出る（こっちがよく使われます）
```

3. なぜ「連想（れんそう）」というの？

「連想」とは、「**Aという言葉から、Bを思い出す**」という意味です。

国と「食べ物・場所」の例

それぞれの国をイメージしたときに、パッと思い浮かぶものはありませんか？

- 日本 から連想するのは... 「お寿司（すし）」
- イタリア から連想するのは... 「ピザ」
- フランス から連想するのは... 「エッフェル塔（とう）」

このように、例えば「**国名（名前）**」を聞いたら、その国の **特徴（データ）** を思い出す」というような心の動きを、「連想」と言います。

「**名前（キー）**」と「**中身（バリュー）**」がセットになっている **仕組み**が、「連想」の仕組みとよく似ているため、「**連想配列**」と呼ばれるのです。

4. 配列と連想配列のまとめ

種類	探し方（呼び方）	どんなときに使う？
普通の配列	0, 1, 2...（数字）	出席番号や買い物リストなど、 順番 が大事なとき
連想配列	"name", "price"...（言葉）	プロフィールや商品の詳細など、 意味 が大事なとき

[!TIP] 学生へのアドバイス：実は「**連想配列**」という言葉は、もともと他のプログラミング言語（PHPやRubyなど）で使われていた言葉なんです。JavaScriptでは、この「**連想配列**」のことを「**オブジェクト**」と呼びます。「名前（キー）と中身（バリュー）がセットになっている」状態は、「プロパティ名とプロパティの値がセットになっている」状態と同じだからです。書き方も同じですね。ただ「**連想配列**」はデータだけでメソッドはありません。

いかがでしょうか？「数字ではなく、名前ですらリスト」だと考えれば簡単ですね！

V. 二次元配列

「**二次元配列（にじげんはいれつ）**」という言葉を知ると、なんだか難しそうに感じますよね。でも大丈夫です。一言でいうと、「**配列の中に、さらに配列が入っているもの**」のことで。

イメージは「マンションの **部屋番号**」

普通の^{はいれつ}配列が「一列にならんだロッカー」だとしたら、二次元^{はいれつ}配列は「たてとよこにならんだマンション」です。

- **1つめの数字**：何階（なんがい）か？
- **2つめの数字**：どの部屋か？

あるいは「**エクセルの表**」をイメージした方がわかりやすいかもしれません。

- **1つめの数字**：行：ぎょう / Row
- **2つめの数字**：列：れつ / Column

1列	2列	3列
1行		
2行		
3行		

JavaScriptでの書きかた

学校のクラスの「座席表（ざせきひょう）」を^{れい}例に見てみましょう。

```
const classMap = [
  ["田中", "鈴木"], // 0番目のグループ（1列目）
  ["佐藤", "マリア"], // 1番目のグループ（2列目）
  ["ジョン", "リー"] // 2番目のグループ（3列目）
];
```

	1列	2列
1行	田中	鈴木
2行	佐藤	マリア
3行	ジョン	リー

中身を取り出すとき

カギカッコを **2つ** `[ ]` 並べて書きます。

```
console.log(classMap[2][1]); // 「リー」さんが出ます！
```

- `classMap[2]` → 3つめのグループ（["ジョン", "リー"]）を選びます。
- `[1]` → その中の、2番目の人（リー）を選びます。

	1列	2列
1行	[0][0]	[0][1]
2行	[1][0]	[1][1]
3行	[2][0]	[2][1]

なぜ「二次元」というの？

- **一次元（いちじげん）**：「線」です。横（よこ）だけにデータがならんでいます。
- **二次元（にじげん）**：「面」です。横（よこ）と縦（たて）にデータが広がっています。

[!TIP] 「**多次元（たじげん）**」って何？ 二次元（たて・よこ）よりもさらに増えて、三次元（奥行き・高さ）など、もっと複雑になったものをまとめて「多次元」と呼びます。

どんなときに使う？

- **ゲームの地図**：どこに宝箱（たからばこ）があるか、座標（ざひょう）で決めるとき。
- **カレンダー**：「○週目の、○曜日」というデータ。
- **写真のデータ**：デジタルの写真は、色のついた点が縦と横に並んでできています。

学生へのアドバイス

二次元配列はいれつを使うとき、一番多いミスはこれです。「**どっちの数字が『たて』で、どっちが『よこ』だけ？**」

混乱（こんらん）したときは、こう教えてあげてください。

「**左側のカッコが『階（かい）』で、右側のカッコが『部屋』だよ！**」

いかがでしょうか？「リストのリスト」という考え方に慣れると、プログラミングでできることが一いっき気に広がります。

VI. オブジェクトはいれつの配列

現実のプログラムでは、以下のように「オブジェクト（情報のセット）」を配列はいれつにまとめることが多いです。これは「顧客名簿めいぼ」や「商品カタログ」のようなイメージです。

```
const customers = [
  { name: "山田", age: 30 },
  { name: "佐藤", age: 25 }
];
```

「**[]**角（かく）カッコ」と「**{ }**波（なみ）カッコ」を使ったこの書き方か かたを、覚えておいてください。